

Laporan Praktikum W8 – Polymorphism

Diajukan untuk memenuhi salah satu tugas praktikum Mata kuliah
Pemrograman Berorientasi Objek



Disusun Oleh:

Daffa Al Ghifari (231511038)

2B-JTK

Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika

Politeknik Negeri Bandung

2024

1. Studi Kasus 1: Another Type Of Employee.

The files Firm.java, Staff.java, StaffMember.java, Volunteer.java, Employee.java, Executive.java, and Hourly.java are from Listings 9.1 – 9.7 in the text. The program illustrates inheritance and polymorphism. In this exercise you will add one more employee type to the class hierarchy (see Figure 9.1 in the text). The employee will be one that is an hourly employee but also earns a commission on sales. Hence the class, which we'll name Commission, will be derived from the Hourly class.

A. Class Commision

```
public class Commission extends Hourly{ 6 usages
    private double totalSales; 5 usages
    private double commisionRate; 2 usages

    public Commission (String eName, String eAddress, String ePhone, String socSecNumber, double rate, double commisionRate) 2 usages
    {
        super (eName, eAddress, ePhone, socSecNumber, rate);
        this.commisionRate = commisionRate;
    }

    public void addSales (double totalSales){ 2 usages
        this.totalSales = this.totalSales + totalSales;
    }

    @Override 3 usages
    public double pay()
    {
        double payment = super.pay() + (totalSales * commisionRate);
        totalSales = 0;
        return payment;
    }

    @Override
    public String toString()
    {
        String result = super.toString();

        result += "\nTotal sales: " + totalSales;

        return result;
    }
}
```

Disini saya membuat sebuah class Commision yang memiliki fitur seperti berikut:

- Commision class ini berperan sebagai subclass dan melakukan extends ke Hourly class yang merupakan superclass.

```
public class Commission extends Hourly{ 6 usages
```

- Commision class ini memiliki dua variabel, yaitu:
 - 1) totalSales bertipe double
 - 2) commisionRate bertipe double, dimana commissionRate ini akan merepersentasikannya dalam bentuk persen, dengan satuan decimal. Contoh 20% maka ditulis 0.2

```
private double totalSales; 5 usages
private double commisionRate; 2 usages
```

- Constructor nya sendiri menggunakan 6 parameter:
 - 1) 5 parameter pertama sama dengan kelas hourly (name, address, phone number, social security number, hourly pay rate), disini kita harus menggunakan Super, dimana berarti disini kita menggunakan atribut yang ada pada superclass, dan dengan kata lain kita telah menerapkan inheritance.

- 2) Dan parameter ke 6 adalah commissionRate dari employee. Kita menggunakan parameter ke 6 ini untuk mengatur commission rate.

```
public Commission (String eName, String eAddress, String ePhone, String socSecNumber, double rate, double commissionRate) 2 usages
{
    super (eName, eAddress, ePhone, socSecNumber, rate);
    this.commissionRate = commissionRate;
}
```

- Terdapat satu method tambahan yaitu:
Public void addSales(doble totalSales) yang dimana parameternya ditambahkan di variabel, yang merepresentasikan total sales.

```
public void addSales (double totalSales){ 2 usages
{
    this.totalSales = this.totalSales + totalSales;
}
```

- Selanjutnya pay method memanggil method pay yang ada di superclass, maka dengan itu kita harus menggunakan override untuk menggantikan fungsi yang sudah ada pada superclass, dan juga menggunakan super pada saat memanggil methodnya. Dan juga total sales harus dikembalikan nilainya menjadi 0. Disini kita tidak perlu lagi mengembalikan nilai hoursWorked Kembali ke 0, karena itu sudah dilakukan pada superclass Hourly.

```
@Override 3 usages
public double pay()
{
    double payment = super.pay() + (totalSales * commissionRate);
    totalSales = 0;
    return payment;
}
```

- Selanjutnya saya memanggil method toString dari superclass tersebut, maka saya menggunakan override saat membuat method toString dan juga menggunakan super untuk memanggil method tersebut dari superclass. Lalu disini saya menambahkan total sales untuk dikonversi dari integer ke string.

```
@Override
public String toString()
{
    String result = super.toString();

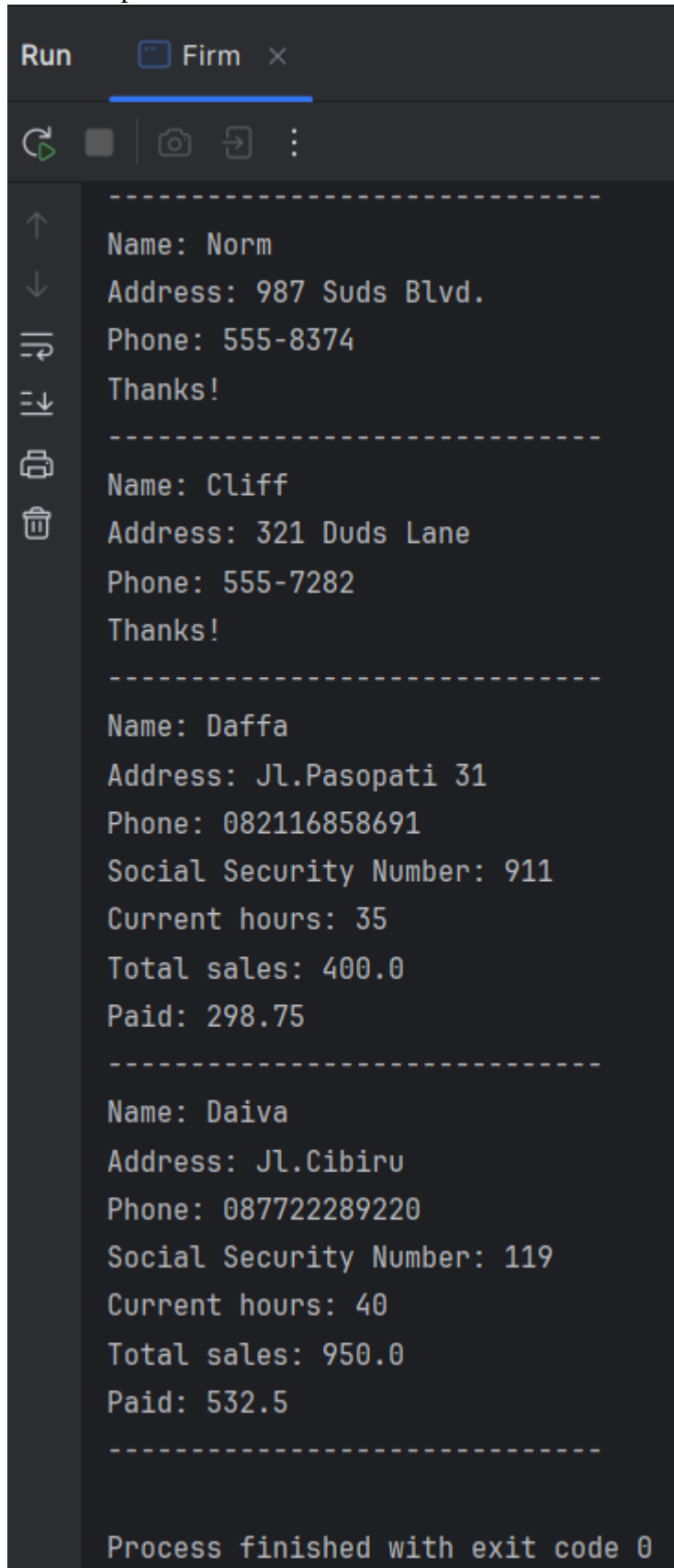
    result += "\nTotal sales: " + totalSales;

    return result;
}
```

B. Penerapan Polymorphism

Penerapan Polymorphism pada soal ini termasuk ke dalam jenis Polymorphic via Inheritance, Inheritance sendiri diterapkan pada penggunaan super saat penggunaan variabel dari superclass di subclass, selanjutnya polymorphic sendiri terjadi saat kita melakukan overriding.

C. Hasil Output



```
Run Firm x
-----
Name: Norm
Address: 987 Suds Blvd.
Phone: 555-8374
Thanks!
-----
Name: Cliff
Address: 321 Duds Lane
Phone: 555-7282
Thanks!
-----
Name: Daffa
Address: Jl.Pasopati 31
Phone: 082116858691
Social Security Number: 911
Current hours: 35
Total sales: 400.0
Paid: 298.75
-----
Name: Daiva
Address: Jl.Cibiru
Phone: 087722289220
Social Security Number: 119
Current hours: 40
Total sales: 950.0
Paid: 532.5
-----
Process finished with exit code 0
```

2. Studi Kasus 2: Painting Shapes

A. Write an abstract class Shape with the following properties: An instance variable shapeName of type String An abstract method area() A toString method that returns the name of the shape

- An instance variable shapeName of type String

```
abstract class Shape { 4 usages 3 inheritors
    public String shapeName; 2 usages
```

- An abstract method area()

```
public abstract double area(); 1 usage 3 implementations
```

- A toString method that returns the name of the shape

```
public String toString() 3 overrides
{
    return "\nname of the shape: " + shapeName;
}
```

B. Sphere.java contains a class for a sphere which is a descendant of Shape. A sphere has a radius and its area (surface area) is given by the formula $4 \cdot \text{PI} \cdot \text{radius}^2$.

```
public class Sphere extends Shape { 2 usages
    private double radius; 4 usages

    public Sphere(double r){ 1 usage
        super(shapeName: "Sphere");
        radius = r;
    }

    @Override 1 usage
    public double area(){
        return 4*Math.PI*radius*radius;
    }

    @Override
    public String toString(){
        return super.toString() + " ,of radius " + radius;
    }
}
```

Disini terdapat penerapan polymorphic via inheritance, inheritance sendiri terjadi Ketika kita melakukan extends ke sebuah superclass, disini class Sphere melakukan extends ke superclass Shape, lalu terdapat penggunaan variabel shapeName dan kita harus menggunakan super untuk memanggilnya, selain itu kita juga menggunakan method dari superclass yaitu method toString(), sehingga kita harus menggunakan super pada saat menggunakannya.

Penerapan polymorphic sendiri pada kode ini terjadi pada saat kita melakukan override pada method area dan toString, override disini berguna untuk menimpa atau mengganti method yang telah ada sebelumnya dengan cara kerja method yang

berbeda. Disini juga kita menambahkan body method karena sebelumnya method area masih berbentuk abstract, dengan begitu kita telah melakukan polymorphic. Pada class Sphere kita mendefinisikan body method dengan rumus Luas Permukaan dari Sphere yaitu: $4 \times \text{PI} \times \text{radius}^2$.

Lalu pada method toString kita ubah untuk menampilkan radius yang kita inputkan.

- C. Rectangle.java, descendants of the Shape class, a rectangle is defined by its length and width and its area is length times width. Define the toString method.

```
public class Rectangle extends Shape { 2 usages
    private double length; 3 usages
    private double width; 3 usages

    public Rectangle(double l, double w) { 1 usage
        super(shapeName: "Sphere");
        length = l;
        width = w;
    }

    @Override 1 usage
    public double area() {
        return length * width;
    }

    @Override
    public String toString() {
        return super.toString() + ",length: " + length + ",width: " + width;
    }
}
```

Disini terdapat penerapan polymorphic via inheritance, inheritance sendiri terjadi Ketika kita melakukan extends ke sebuah superclass, disini class Rectangle melakukan extends ke superclass Shape, lalu terdapat penggunaan variabel shapeName dan kita harus menggunakan super untuk memanggilnya, selain itu kita juga menggunakan method dari superclass yaitu method toString(), sehingga kita harus menggunakan super pada saat menggunakannya.

Penerapan polymorphic sendiri pada kode ini terjadi pada saat kita melakukan override pada method area dan toString, override disini berguna untuk menimpa atau mengganti method yang telah ada sebelumnya dengan cara kerja method yang berbeda. Disini juga kita menambahkan body method karena sebelumnya method area masih berbentuk abstract, dengan begitu kita telah melakukan polymorphic.

Pada class Rectangle kita mendefinisikan body method dengan rumus Luas dari Persegi yaitu: $\text{length} \times \text{width}$

Lalu pada method toString kita ubah untuk menampilkan length dan width yang kita masukan.

- D. Cylinder.java, descendants of the Shape class, a cylinder is defined by a radius and height and its area (surface area) is $\text{PI} \times \text{radius}^2 \times \text{height}$. Define the toString method.

```

public class Cylinder extends Shape{ 2 usages
    private double radius; 4 usages
    private double height; 3 usages

    public Cylinder(double r, double h){ 1 usage
        super(shapeName: "Cylinder");
        radius = r;
        height = h;
    }

    @Override 1 usage
    public double area() {
        return Math.PI*radius*radius*height;
    }

    @Override
    public String toString() {
        return super.toString() + ",radius: " + radius + ",height: " + height;
    }
}

```

Disini terdapat penerapan polymorphic via inheritance, inheritance sendiri terjadi Ketika kita melakukan extends ke sebuah superclass, disini class Cylinder melakukan extends ke superclass Shape, lalu terdapat penggunaan variabel shapeName dan kita harus menggunakan super untuk memanggilnya, selain itu kita juga menggunakan method dari superclass yaitu method toString(), sehingga kita harus menggunakan super pada saat menggunakannya.

Penerapan polymorphic sendiri pada kode ini terjadi pada saat kita melakukan override pada method area dan toString, override disini berguna untuk menimpa atau mengganti method yang telah ada sebelumnya dengan cara kerja method yang berbeda. Disini juga kita menambahkan body method karena sebelumnya method area masih berbentuk abstract, dengan begitu kita telah melakukan polymorphic.

Pada class Rectangle kita mendefinisikan body method dengan rumus Luas Permukaan dari Cylinder yaitu: $\text{PI} \times \text{radius}^2 \times \text{height}$.

Lalu pada method toString kita ubah untuk menampilkan radius dan height yang kita masukan.

- E. The file Paint.java contains a class for a type of paint (which has a "coverage" and a method to compute the amount of paint needed to paint a shape). Correct the return statement in the amount method so the correct amount will be returned. Use the fact that the amount of paint needed is the area of the shape divided by the coverage for the paint. (NOTE: Leave the print statement - it is there for illustration purposes, so you can see the method operating on different types of Shape objects.)

```

public class Paint { 2 usages
    private double coverage; 2 usages

    public Paint(double c){ 1 usage
        coverage = c;
    }

    public double amount(Shape s){ 3 usages
        System.out.println("Computing amount for " + s);
        return s.area()/coverage;
    }
}

```

Disini kita diperintahkan untuk memodifikasi class Paint, saya melakukan perubahan pada hasil return pada method amount, dimana jumlah cat yang dibutuhkan adalah permukaan dari shape dibagi coverage cat. Sehingga saya menuliskan seperti berikut

```
return s.area()/coverage
```

- F. The file PaintThings.java contains a program that computes the amount of paint needed to paint various shapes. A paint object has been instantiated. Add the following to complete the program:

```
public class PaintThings {
    public static void main (String[] args){
        final double COVERAGE = 350;
        Paint paint = new Paint (COVERAGE);

        Rectangle deck = new Rectangle( l: 20, w: 35);
        Sphere bigBall = new Sphere( r: 15);
        Cylinder tank = new Cylinder( r: 10, h: 30);

        double deckAmt, ballAmt, tankAmt;

        deckAmt = paint.amount(deck);
        ballAmt = paint.amount(bigBall);
        tankAmt = paint.amount(tank);

        DecimalFormat fmt = new DecimalFormat( pattern: "0.#");
        System.out.println("\nNumber of gallons of paint needed ...");
        System.out.println("Deck " + fmt.format(deckAmt));
        System.out.println("Big Ball " + fmt.format (ballAmt));
        System.out.println("Tank " + fmt.format(tankAmt));
    }
}
```

- Instantiate the three shape objects: deck to be a 20 by 35 foot rectangle, bigBall to be a sphere of radius 15, and tank to be a cylinder of radius 10 and height 30.

```
Rectangle deck = new Rectangle( l: 20, w: 35);
Sphere bigBall = new Sphere( r: 15);
Cylinder tank = new Cylinder( r: 10, h: 30);
```

Disini kita membuat 3 objek yaitu Rectangle dengan nama deck dengan length 20 dan wight 35, Sphere dengan nama bigBall dengan radius 15, dan Cylinder dengan nama Tank dengan radius 10, dah height 30.

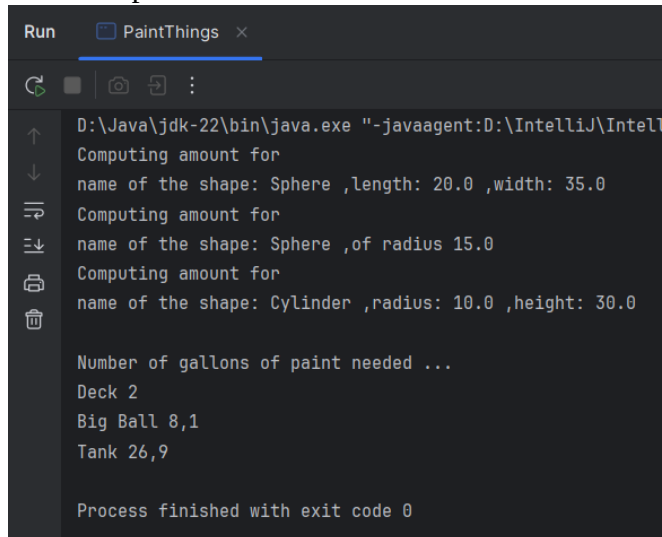
- Make the appropriate method calls to assign the correct values to the three amount variables.

```
double deckAmt, ballAmt, tankAmt;

deckAmt = paint.amount(deck);
ballAmt = paint.amount(bigBall);
tankAmt = paint.amount(tank);
```

Disini kita memanggil method amount yang ada pada class paint, sehingga cara pemanggilannya adalah paint.amount(parameter). Parameternya sendiri kita masukkan objek yang kita buat sebelumnya, dengan begitu akan terjadi proses polymorphic pada perhitungan cat yang dibutuhkan pada deck, bigball, dan tank sesuai dengan bentuk bangunan tersebut dan juga dengan atribut yang diatur.

G. Hasil Output



```
Run PaintThings x
D:\Java\jdk-22\bin\java.exe "-javaagent:D:\IntelliJ\IntelliJ
Computing amount for
name of the shape: Sphere ,length: 20.0 ,width: 35.0
Computing amount for
name of the shape: Sphere ,of radius 15.0
Computing amount for
name of the shape: Cylinder ,radius: 10.0 ,height: 30.0

Number of gallons of paint needed ...
Deck 2
Big Ball 8,1
Tank 26,9

Process finished with exit code 0
```

H. Penerapan Polymorphism

Pada program diatas, itu menerapkan polymorphism via inheritance

- Implementasi Spesifik dengan Pemanggilan Polymorphic: Ketika metode `amount()` dipanggil dengan objek `Sphere`, `Rectangle`, atau `Cylinder`, metode tersebut tetap dapat bekerja dengan objek tersebut secara polymorphic. Artinya, meskipun objek tersebut memiliki implementasi `area()` yang berbeda-beda, cara pemanggilan metode tersebut tetap seragam karena mereka semua dianggap sebagai tipe `Shape`.
- Pemanggilan Metode dengan Objek yang Berbeda: Saat objek `Shape` diproses oleh metode `amount` dalam kelas `Paint`, penerapan polymorphism memungkinkan metode tersebut untuk menerima objek apapun dari tipe turunan `Shape`. Ini memperlihatkan fleksibilitas di mana metode yang sama dapat bekerja dengan objek yang berbeda, tergantung pada bentuk (seperti `Sphere`, `Rectangle`, `Cylinder`), dan menghitung area atau volume berdasarkan definisi masing-masing.

3. Studi Kasus 3: Polymorphic Sorting

- A. The file Numbers.java reads in an array of integers, invokes the selection sort algorithm to sort them, and then prints the sorted array. Save Sorting.java and Numbers.java to your directory. Numbers.java won't compile in its current form. Study it to see if you can figure out why.

```
import java.util.Scanner;

public class Numbers {
    public static void main(String[] args){
        Integer[] intList;
        int size;

        Scanner scan = new Scanner(System.in);

        System.out.print("\nHow many integers do you want to sort? ");
        size = scan.nextInt();
        intList = new Integer[size];

        System.out.println("\nEnter the numbers...");
        for (int i = 0; i < size; i++) {
            intList[i] = scan.nextInt();
        }

        Sorting.insertionSort(intList);
        System.out.println("\nYour numbers in sorted order...");
        for (int i = 0; i < size; i++){
            System.out.print(intList[i] + " ");
            System.out.println();
        }
    }
}
```

- B. Try to compile Numbers.java and see what the error message is. The problem involves the difference between primitive data and objects. Change the program so it will work correctly (note: you don't need to make many changes - the autoboxing feature of Java 1.5 will take care of most conversions from int to Integer).

Disini terdapat error yang saya alami, masalahnya terjadi karena perbedaan penggunaan tipe data antara primitive dan objek, pada array intList sebelumnya menggunakan tipe data int, yaitu tipe data primitive, namun pada saat melakukan scan (scan.nextInt) yang dilakukan scan adalah Integer yaitu tipe data Integer yang dijadikan objek. Sehingga terjadilah error.

Untuk mengatasi masalah diatas, saya perlu mengubah tipe data dari array intList yang tadinya menggunakan tipe data int primitive menjadi tipe data Integer untuk menjadikan intList adalah objek Integer.

- C. Write a program Strings.java, similar to Numbers.java, that reads in an array of String objects and sorts them. You may just copy and edit Numbers.java.

```

import java.util.Scanner;

public class Strings {
    public static void main(String[] args){
        String[] stringList;
        int size;

        Scanner scan = new Scanner(System.in);

        System.out.print("\nHow many strings do you want to sort? ");
        size = scan.nextInt();
        stringList = new String[size];

        System.out.println("\nEnter the strings...");
        for (int i = 0; i < size; i++) {
            stringList[i] = scan.next(); // Read strings
        }

        Sorting.insertionSort(stringList); // Sorts String[]
        System.out.println("\nYour strings in sorted order...");
        for (int i = 0; i < size; i++){
            System.out.print(stringList[i] + " ");
            System.out.println();
        }
    }
}

```

Disini saya diperintahkan untuk membuat file Strings.java yang mengimplementasikan cara yang sama pada Numbers.java, namun sekarang kita akan membaca sebuah String bukan Number.

Saya mengganti tipe data stringList yang tadinya Integer menjadi String, dan pada scan saya menggantinya yang tadinya scan.nextInt, menjadi scan.next() agar dapat membaca input berupa String.

- D. Modify the insertionSort algorithm so that it sorts in descending order rather than ascending order.

```

public static void insertionSort (Comparable[] list) { 3 usages
    for (int index = 1; index < list.length; index++) {
        Comparable key = list[index];
        int position = index;

        while (position > 0 && key.compareTo(list[position - 1]) > 0) {
            list[position] = list[position - 1];
            position--;
        }
        list[position] = key;
    }
}

```

Disini saya diperintahkan untuk memodifikasi algoritma method insertionSort yang tadinya mengurutkan dari yang terkecil ke terbesar (ascending) menjadi dari yang terbesar ke terkecil (descending). Disini yang saya lakukan adalah hanya tinggal mengganti kondisi `key.compareTo(list[position - 1]) < 0`, menjadi

key.compareTo(list[position - 1]) > 0. Disini kita akan membandingkannya dengan urutan descending dengan reversing atau membalikan compareTo. Dengan begitu algoritma ini akan mengurutkan dengan Descending.

E. Change Numbers.java and Strings.java to call insertionSort rather than selectionSort. Run both to make sure the sorting is correct.

- Numbers.java

```
import java.util.Scanner;

public class Numbers {
    public static void main(String[] args){
        Integer[] intList;
        int size;

        Scanner scan = new Scanner(System.in);

        System.out.print("\nHow many integers do you want to sort? ");
        size = scan.nextInt();
        intList = new Integer[size];

        System.out.println("\nEnter the numbers...");
        for (int i = 0; i < size; i++) {
            intList[i] = scan.nextInt();
        }

        Sorting.insertionSort(intList);
        System.out.println("\nYour numbers in sorted order...");
        for (int i = 0; i < size; i++){
            System.out.print(intList[i] + " ");
            System.out.println();
        }
    }
}
```

- Strings.java

```
import java.util.Scanner;

public class Strings {
    public static void main(String[] args){
        String[] stringList;
        int size;

        Scanner scan = new Scanner(System.in);

        System.out.print("\nHow many strings do you want to sort? ");
        size = scan.nextInt();
        stringList = new String[size];

        System.out.println("\nEnter the strings...");
        for (int i = 0; i < size; i++) {
            stringList[i] = scan.next(); // Read strings
        }

        Sorting.insertionSort(stringList); // Sorts String[]
        System.out.println("\nYour strings in sorted order...");
        for (int i = 0; i < size; i++){
            System.out.print(stringList[i] + " ");
            System.out.println();
        }
    }
}
```

Disini kita diperintahkan untuk mengganti Numbers.java dan Strings.java yang tadinya memanggil method selectionSort menjadi memanggil method insertionSort.

Disini saya hanya perlu mengganti `Sorting.selectionSort(intList)` menjadi `Sorting.insertionSort`. Saya lakukan hal tersebut pada kedua file yang diperintahkan.

- F. The file `Salesperson.java` partially defines a class that represents a sales person. This is very similar to the `Contact` class in Listing 9.10. However, a sales person has a first name, last name, and a total number of sales (an `int`) rather than a first name, last name, and phone number. Complete the `compareTo` method in the `Salesperson` class. The comparison should be based on total sales; that is, return a negative number if the executing object has total sales less than the other object and return a positive number if the sales are greater. Use the name of the sales person to break a tie (alphabetical order).

```
public int compareTo(Salesperson other) {
    if (this.totalSales != other.totalSales) {
        return Integer.compare(other.totalSales, this.totalSales);
    }

    int lastNameComparison = other.lastName.compareTo(this.lastName);
    if (lastNameComparison != 0) {
        return lastNameComparison;
    }

    return other.firstName.compareTo(this.firstName);
}
```

Disini pada method `compareTo` saya melakukan modifikasi sesuai perintah pada soal, berikut penjelasannya

`Integer.compare(this.totalSales, other.totalSales)` membandingkan total penjualan penjual satu dengan penjual lainnya.

- Jika total sales berbeda, maka method tersebut akan return nilai dari `Integer.compare(other.totalSales, this.totalSales)`. Dimana penerapan mengembalikan angka negative jika total sales dari suatu objek kurang dari objek lainnya dan mengembalikan angka positif jika total sales lebih dari objek lainnya? Disini saya menggunakan `Integer.compare()` dimana method tersebut akan mengembalikan nilai sebagai berikut

- Angka positif jika `other.totalSales > this.totalSales`.
- Angka negative jika `other.totalSales < this.totalSales`.

```
if (this.totalSales != other.totalSales) {
    return Integer.compare(other.totalSales, this.totalSales);
}
```

- Selanjutnya untuk break a tie, yang maksudnya adalah jika ada dua total sales ada yang sama, maka pemilihan urutan berikutnya akan ditentukan berdasarkan urutan nama berdasarkan alfabet.

```
int lastNameComparison = other.lastName.compareTo(this.lastName);
if (lastNameComparison != 0) {
    return lastNameComparison;
}
```

Jika dua salesperson memiliki total penjualan yang sama, kita bisa menggunakan nama belakang atau nama depan mereka untuk menentukan siapa

yang ditempatkan lebih dahulu berdasarkan urutan abjad. Dalam hal ini. Nama yang lebih awal dalam alfabet (misalnya, "Adams" sebelum "Brown") akan ditempatkan lebih dulu jika menggunakan urutan alfabet normal.

- Jika nama akhir masih sama, maka nama awal akan di bandingkan terbalik

```
return other.firstName.compareTo(this.firstName);  
}
```

G. The file WeeklySales.java contains a driver for testing the compareTo method and the sorting (this is similar to Listing 9.8 in the text). Compile and run it. Make sure your compareTo method is correct. The sales staff should be listed in order of sales from most to least with the four people having the same number of sales in reverse alphabetical order.

The screenshot shows the IntelliJ IDEA IDE with the 'WeeklySales.java' file open. The code defines a 'Salesperson' class with 'first', 'last', and 'sales' attributes, and a 'WeeklySales' class with a 'main' method that creates an array of 10 sales staff and sorts them using 'insertionSort'. The output window shows the results of the sorting, listing sales staff in descending order of sales, with those having the same sales amount listed in reverse alphabetical order.

```
public class WeeklySales {  
    public static void main(String[] args){  
        Salesperson[] salesStaff = new Salesperson[10];  
  
        salesStaff[0] = new Salesperson( first: "Jane", last: "Jones", sales: 3000);  
        salesStaff[1] = new Salesperson( first: "Daffy", last: "Duck", sales: 4935);  
        salesStaff[2] = new Salesperson( first: "James", last: "Jones", sales: 3000);  
        salesStaff[3] = new Salesperson( first: "Dick", last: "Walter", sales: 2800);  
        salesStaff[4] = new Salesperson( first: "Don", last: "Trump", sales: 1570);  
        salesStaff[5] = new Salesperson( first: "Jane", last: "Black", sales: 3000);  
        salesStaff[6] = new Salesperson( first: "Harry", last: "Taylor", sales: 7300);  
        salesStaff[7] = new Salesperson( first: "Andy", last: "Adams", sales: 5000);  
        salesStaff[8] = new Salesperson( first: "Jim", last: "Doe", sales: 2850);  
        salesStaff[9] = new Salesperson( first: "Walt", last: "Smith", sales: 3000);  
  
        Sorting.insertionSort(salesStaff);  
  
        System.out.println("\nRanking of Sales for the Week\n");  
  
        for (Salesperson s : salesStaff)  
            System.out.println(s);  
    }  
}
```

Ranking of Sales for the Week

Taylor, Harry:	7300
Adams, Andy:	5000
Duck, Daffy:	4935
Black, Jane:	3000
Jones, James:	3000
Jones, Jane:	3000
Smith, Walt:	3000
Doe, Jim:	2850
Walter, Dick:	2800
Trump, Don:	1570

Process finished with exit code 0

H. Penerapan Polymorphism

- Interface Comparable: Dalam program diatas, kelas Salesperson implement interface Comparable. Ini berarti setiap objek Salesperson dapat dibandingkan dengan objek Salesperson lainnya menggunakan metode compareTo. Pada dasarnya, meskipun objek-objek ini mungkin berbeda secara spesifik, mereka diperlakukan sama selama mereka mendukung operasi compareTo.

```
public class Salesperson implements Comparable<Salesperson>
```

Polymorphism terjadi ketika saya memanggil metode compareTo dari objek Salesperson pada proses pengurutan. Misalnya, ketika saya menggunakan metode `Sorting.insertionSort(salesStaff);`, insertionSort dapat bekerja pada array Salesperson meskipun Sorting mungkin dirancang untuk bekerja pada berbagai tipe objek yang mengimplementasikan Comparable.

- `Sorting.insertionSort(Comparable[] list):` Metode insertionSort diimplementasikan dengan polymorphism karena dapat bekerja dengan array apa pun yang elemennya mengimplementasikan antarmuka Comparable. Metode ini tidak perlu tahu tipe objek spesifik apa yang sedang diurutkan selama objek-objek tersebut mengimplementasikan antarmuka Comparable dan menyediakan logika perbandingan mereka sendiri melalui compareTo.

```
public static void insertionSort(Comparable[] list)
```

Dalam program diatas, Salesperson yang mengimplementasikan Comparable dapat diteruskan ke insertionSort, dan objek tersebut akan dibandingkan menggunakan implementasi compareTo dari Salesperson.

Link Github

[daffalghi/PBO-Daffa-Al-Ghifari \(github.com\)](https://github.com/daffalghi/PBO-Daffa-Al-Ghifari)